店智 Copilot 技术文档

> 面向中小电商运营者的 AI Agent 智能助手
> 版本：v1.0 · 2026-10-24

---

1. 项目概述

1.1 问题背景

中小电商卖家日常面临三类高频痛点：

1. **规则查询难**：平台规则（保证金、处罚标准、响应时效）分散且频繁更新，靠经验记忆易踩坑；
2. **数据解读门槛高**：有后台数据但不会诊断——转化率低该先修什么、流量结构是否健康，缺少分析方法；
3. **文案工作重复**：商品标题、卖点、客服话术、活动文案，量大且格式固定，消耗人力。

现有通用聊天机器人无法解决这些问题：答案不可溯源（无法核对依据）、不懂店铺数据（只能泛泛而谈）、过程不透明（用户不知道 AI 是怎么得出结论的）。

1.2 方案定位

店智 Copilot 是一个**会查数、会引用、会思考**的电商运营 AI Agent 助手：

能力	实现	用户价值
RAG 知识库问答	BM25+向量混合检索，回答带 [1][2] 引用角标	答案可溯源，未覆盖时明确说"未覆盖"而非编造
多步 Agent 推理	手写 ReAct 循环，自主调用 4 类工具	像人类运营一样"先查数、再分析、后建议"

能力	实现	用户价值
工具过程可视化	步骤卡片实时展示每次工具调用	透明可审计，建立用户信任
多模态输入	商品图→GLM-4V 生成文案；语音输入	降低输入门槛，提升文案生产效率
MCP 开放生态	工具以 MCP 协议标准化暴露	可被 Claude Desktop / Cursor 等任意 MCP 客户端复用

1.3 与普通聊天机器人的区别

- **可溯源**：每条知识库回答标注引用编号，点击可核对原文；
- **会动手**：不只生成文字，还能查询结构化店铺数据、执行合规检查；
- **过程透明**：Agent 的每次工具调用以步骤卡片实时呈现；
- **不编造**：检索无结果时明确声明"知识库未覆盖"，而非凭空生成。

---

2. 系统架构

2.1 分层架构图

```
浏览器 (Next.js)
  |  流式对话 + Agent 步骤卡片 + 引用角标[1] + 语音/图片
  ▼
FastAPI  backend/app/          ← AI 能力唯一接入层
  ├── api/routes                 # /v1/chat/completions（RAG问答）
  |                              # /v1/agent/chat（Agent SSE）
  |                              # /v1/vision/caption（GLM-4V 图生文案）
  ├── agent/                     # 手写 ReAct 循环 + 工具注册表
  |    ├── loop                  #   DeepSeek function calling，最多 5 步
  |    └── tools                 #   kb_search / data_query / copywriting / policy_check
  ├── rag/                       # 混合检索（jieba BM25 + 向量 RRF 融合）
  |    ├── pipeline              #   分块 → fastembed 嵌入 → LanceDB
  |    ├── retrieve              #   检索 API
  |    ├── selftest              #   检索质量自测
  |    └── eval / eval_v2        #   开/关 RAG 对比评测
  └── mcp_server/                # 独立 MCP 进程（stdio），复用 agent 工具
```

```
▼
外部: DeepSeek API • GLM-4V（可选）
```

核心不变式：前端永远不直连任何模型 API——一切 AI 请求走 FastAPI，密钥校验与注入全部在 Python 层完成（符合比赛规则"后端服务仅作为 AI 能力的接入层"）。

2.2 关键架构决策

#	决策	理由
D1	**手写 ReAct 循环**，不用 LangChain/LangGraph	工具仅 4 个，框架是负资产；手写循环比赛里讲得清、可完全控制 SSE 事件协议
D2	`backend/app/` 分层包结构	config/prompts/agent/rag/services 职责清晰，M2 加工具后不混乱
D3	混合检索 RRF 融合 + 引用角标	BM25+向量是多项目验证的共识方案；引用是 RAG 可信度的关键
D4	MCP 独立进程（FastMCP stdio）	标准化暴露工具，生态互通；独立进程不与 FastAPI 端口冲突
D5	Agent 步骤可视化	差异化体验；trace JSON → UI 卡片，实现成本低
D6	评测 v2 引用有据性评分	v1 发现裸模型常识题已很强，RAG 优势须在"防编造+可溯源"维度评估

2.3 一次 Agent 请求的完整时序

```
用户输入 → Next.js /api/agent（透传） → FastAPI /v1/agent/chat
  → ReAct 循环：
      1. messages = [system, history, user]
      2. 调 DeepSeek（带 tools schema）
      3. 若返回 tool_calls → 执行工具 → SSE 发 step/step_result 事件 → 回填 tool message → 回到 2
      4. 若返回 content → SSE 发 delta 事件（流式） → 发 done 事件
  → Next.js 原样透传 SSE → 前端 agentClient 解析 → 步骤卡片 + 文本渲染
```

---

3. RAG 知识库

3.1 语料

- 40 篇种子文档（`data/raw/*.md`），覆盖 10 个类目 × 4 篇： 平台规则、选品、标题与搜索、付费推广、内容与直播、客服与售后、数据指标、活动促销、物流仓储、账号与体验分
- 每篇带 YAML frontmatter（title / source / url / category），便于溯源

3.2 处理管线

```
原始 .md → frontmatter 解析 → 文本清洗 → 分块（目标 800 字符，重叠 200）
  → fastembed BGE-small-zh-v1.5 本地嵌入（512 维，HF 镜像加速）
  → LanceDB 持久化（data/index/local/chunks.lance）
```

支持 `EMBED_PROVIDER=siliconflow` 切换至 BGE-M3 云嵌入（更大上下文）。

3.3 混合检索

- **向量通道**：查询嵌入 → 余弦相似度 top-k
- **关键词通道**：jieba 分词（带 bigram 兜底）→ 自实现 BM25
- **融合**：RRF（Reciprocal Rank Fusion，k=60），默认返回 top-4

3.4 问答接入

命中知识库时，检索片段以带编号的 system 上下文注入：

```
[1] 《DSR 评分》（分类：平台规则常识）…
[2] 《账号与体验分》…
要求：引用依据时在句末标注 [1][2] 角标；如果检索结果不足以回答，
请明确说明"知识库未覆盖"，不要编造。
```

`body.rag=false` 可按请求关闭（评测脚本用）。

3.5 评测结果

v1（20 问通用对比，0/1/2 分制，满分 40）：有 RAG 26 vs 无 RAG 23
→ 详见 `docs/rag-eval.md`。发现：裸模型对通用常识题已能答对，RAG 增量有限。

v2（15 问引用有据性评分，满分 30）：有 RAG 27 vs 无 RAG 30
→ 详见 `docs/rag-eval-v2.md`。失败 3 题均为语料未覆盖的细节：

- RAG 组诚实回答"知识库未覆盖"并引用相关原则 → 行为正确（防编造），裁判按"无确切答案"给 1 分；
- 无 RAG 组靠预训练常识答出具体数字 → 裁判给 2 分。

结论（诚实呈现）：

1. 对公开常识，裸模型已足够强，RAG 增量有限；
2. RAG 的真正价值在于：**私有/新鲜数据**（语料外知识模型答不出）、**防编造**（宁可说"未覆盖"也不 hallucinate）、**可溯源引用**（[1][2] 角标可核对原文）；
3. 竞赛演示建议：用语料外的私有数据对比开关 RAG，才能拉开差距；
4. 语料扩充方向：优先真实平台规则原文（数值型细节），而非方法论文章。

检索质量自测（C9）：10 条典型查询 top-3 相关率 9/10 = 90%（阈值 ≥80%）。

---

4. Agent 内核

4.1 ReAct 循环

```
输入 → DeepSeek function calling（带 4 个工具 schema）
  ├── 返回 tool_calls → 逐个执行 → SSE: step / step_result → 回填 → 循环（最多 5 步）
  └── 返回 content → SSE: delta（流式） → done
```

- MAX_STEPS=5，防失控；超限返回明确错误提示
- 工具异常不抛出，转为可读错误字符串回填给模型（模型可自我修正）
- SSE 事件协议：`step` / `step_result` / `delta` / `done` / `error`

4.2 工具规格

工具	参数	返回	说明
`kb_search`	query	JSON：top-3 片段（title/category/text）	复用 RAG 混合检索
`data_query`	query	JSON：GMV/订单/访客/转化率/退款率/热销品/流量来源	P1 为样例数据，预留真实数据源接入

工具	参数	返回	说明
`copywriting`	product_name, selling_points	JSON：title / short_desc / bullet_points	商品文案生成
`policy_check`	scenario	JSON：risk_level + suggestion	场景合规检查（内部走 kb_search）

工具以 `Tool` dataclass 注册到全局 REGISTRY，同一实现同时服务： - ReAct 循环（HTTP 场景） - MCP server（生态复用场景）

4.3 实测案例

输入：「查店铺今日数据并给建议」

```
STEP data_query args={'query': '今日GMV、订单量、访客数、转化率、退款率'}
RESULT data_query: {"date": "2026-10-23", "data": {"gmv": 12840.5, ...}}
STEP data_query args={'query': '今日热销商品和流量来源'}
RESULT data_query: {...}
[138 段流式文本] → 完整分析报告（数据表格 + 6 条解读建议 + 优先动作清单）
DONE
```

Agent 自主决定两次调用数据工具（先总体指标、后热销品与流量），再综合生成报告——与人类运营的分析路径一致。

4.4 安全边界

- 密钥仅存于 `backend/.env`（gitignore），前端零暴露；
- 工具参数经 JSON Schema 校验，多余/缺失参数返回可读错误而非崩溃；
- DeepSeek 上游错误透传为带中文说明的 JSON，前端优雅展示。

---

5. 多模态与 MCP

5.1 GLM-4V 图生文案

- 端点：`POST /v1/vision/caption`，支持 `image_url` 或 `image_base64`
- 请求 GLM-4V-Flash，提示词约束返回 JSON（title / selling_points / description）

- 解析兜底：正则提取 `{...}` 块，失败时返回原文
- 降级：未配置 `VISION_API_KEY` 时返回 400 中文提示（功能可选，不阻塞主链路）
- 前端：上传图片 → 识别结果卡片 → 标题自动填入输入框（可编辑后发送）

5.2 语音输入

浏览器原生 Web Speech API（Chrome/Edge），`zh-CN` 识别，文字直接写入输入框。零服务端成本。

5.3 MCP Server

- 独立进程：`python -m backend.mcp_server`（FastMCP，stdio 传输）
- 暴露 4 个工具（与 Agent 内核共享实现）
- 验证：`npx @modelcontextprotocol/inspector python -m backend.mcp_server`
- 意义：工具可被任何 MCP 客户端（Claude Desktop、Cursor 等）复用，一次开发，生态互通

---

6. 会话管理与引用溯源（P4）

6.1 数据模型与 API 契约

- 会话存储：SQLite（`data/app.db`，stdlib `sqlite3` autocommit，不入库）
- `sessions` 表：`id TEXT PK`（`uuid4().hex[:12]`）· `title` · `mode`（`rag` / `agent`）· `messages TEXT`（JSON 全量）· `created_at` / `updated_at REAL`（epoch 秒浮点）
- API（前端经 catch-all 代理 `app/api/v1/[...path]/route.ts` → `BACKEND_URL`，浏览器永不直连 8000）：

方法	路径	说明
GET	`/v1/sessions`	元数据列表，`updated_at` 降序置顶
POST	`/v1/sessions`	新建（可带初始 mode/title）
GET	`/v1/sessions/{id}`	完整消息恢复
PUT	`/v1/sessions/{id}`	全量覆盖 messages

方法	路径	说明
PATCH	`/v1/sessions/{id}`	`{title?, mode?}` 至少一个字段
DELETE	`/v1/sessions/{id}`	删除（前端两步确认后才发）
GET	`/v1/rag/search?q=&k=4`	检索溯源，与生成端**同序同参** → 角标 [n] 可回查原文

6.2 前端状态机（`lib/useSessions.ts`）

会话切换是典型的「异步竞态陷阱」，用四层防护：

1. **debounce 1.5s**：消息变化 `PUT` 全量覆盖，切会话/新建前强制 flush；
2. **签名守卫**：`signature(id, messages)` 存入 `savedSigRef`，发送前比对，相同则跳过——杜绝重复写与「旧签名覆盖新内容」；
3. **loadingRef**：加载窗口内禁止 flush，防止「旧会话的消息写进新会话」；
4. **activeIdRef 镜像**：回调闭包里读 ref 而非 state，规避闭包过期。

幽灵会话排查结论（诚实呈现）：验收期曾出现一条来源不明的会话。处置：删除后**干净重放全部流程 + Playwright 网络抓包（filter api/v1）零异常 POST**；代码审查确认无自动创建路径。判定为探索阶段残留/环境偶发，非系统性 bug——记录于此供评审复核。

6.3 引用溯源链路

```
RAG 回答（DeepSeek 生成，system 注入带编号片段）
  → 前端 MarkdownText 将 [1][2] 渲染为可点角标
  → 点击 → CitationCard 右下弹出
  → 调 GET /v1/rag/search?q=<原问题>&k=4
  → 与生成端完全同序的 top-4 片段，按角标序号定位原文
```

「生成用什么，回查就返回什么」——同端点、同参数、同排序，避免了生成后溯源的二次检索不一致问题。

6.4 全链路验收（e2e）

Playwright + API 双通道，12/12 通过：刷新恢复 / 角标溯源 / 追问仅 PUT / 改名 PATCH / 两步删除取消+确认 / 空会话切模式 PATCH×2 / Agent 模式 data_query 步骤卡片 + SQLite 真数据 + 自动标题。终验：`p4_api_test.py` 12/12，tsc=0，eslint=0，next build=0，测试会话已清空。

7. 工程质量

6.1 验收清单

验收项	标准	结果	证据
类型检查 / Lint / Build	退出码 0	✅	tsc + eslint + next build
索引构建可复现（C8）	从零重建成功	✅	`pipeline build` → 40 docs → 40 chunks, dim=512
检索质量（C9）	top-3 相关率 ≥80%	✅	selftest 9/10 = 90%
RAG 效果可证明（C10）	有据评测	✅	rag-eval.md + rag-eval-v2.md
引用可溯源（C11）	回答含 [1] 角标	✅	实测 citation=True
Agent 多步调用（C13）	工具调用真实发生	✅	steps=2, deltas=138, done=True
MCP 可独立调用（C14）	Inspector 调通	✅	FastMCP 4 tools

6.2 目录结构

```
backend/
  app/
    main.py             # FastAPI 入口：三个 AI 端点
    agent/
      loop.py           # 手写 ReAct 循环
      tools/__init__.py # Tool dataclass + 注册表
      __init__.py       # 4 个工具实现
    rag/
      pipeline.py       # 语料清洗/分块/嵌入/索引
```

```
      retrieve.py       # 混合检索
      selftest.py       # C9 自测
      eval.py           # C10 v1 评测
      eval_v2.py        # C10 v2 引用有据性评测
  mcp_server/__init__.py
  requirements.txt
  .env.example
data/
  raw/                  # 40 篇种子语料
  index/                # LanceDB 索引（构建产物）
app/                    # Next.js
  api/{chat,agent,vision}/route.ts
components/
  ChatWindow.tsx        # 双模式聊天 + 语音/图片
  AgentSteps.tsx        # Agent 步骤卡片
  MessageBubble.tsx
lib/
  ai.ts                 # AI provider（指向 FastAPI）
  agentClient.ts        # Agent SSE 解析器
docs/
  tech-plan.md          # 技术对比与架构决策
  rag-eval.md / v2      # 评测报告
  video-script.md       # 演示视频脚本
  tech-doc-outline.md   # 本文档大纲
```

6.3 密钥与错误处理

- `.env` / `backend/.env` / `data/index/` 均 gitignore，仓库零密钥；
- 上游 API 错误统一返回 `{error: {message: "中文说明"}}`，前端内联展示不白屏；
- Agent SSE 单题失败不影响整体（评测脚本内置指数退避重试）。

---

8. 扩展设计：长会话与上下文压缩

> 本章为**设计储备**（未实现，未计入验收）。目的：回答评审"会话长了怎么办"，并给出与业界标杆的对照。

8.1 现状与边界

本项目采用**全量重放**：每轮将该会话全部 `{role, content}` 发给模型，SQLite 持久化。在演示规模（单会话几十轮内）简单可靠，零额外失败模式。真实产品中数百轮会话会撞上 200k 上下文窗口，需要压缩。

8.2 业界标杆：Claude Code 的五种压缩机制

对 Claude Code 反混淆源码（社区开源还原版）与官方文档的调研结论：

机制	触发	调 LLM?	行为
Auto full compact	单轮后 token ≥ 阈值	✅	全历史替换为摘要，保留近期尾部
手动 `/compact`	用户命令	✅	同上，可附摘要侧重点
Sub-agent compact	子代理回合前	✅	对子代理自身历史压缩
Microcompact	token 紧张	❌	仅清旧工具结果，零成本
Session memory compact	自动	❌	用落盘会话记忆文件当摘要

关键阈值公式（200k 窗口示例）：

```
有效窗口 = context_window - min(max_output, 20000)   // 为摘要输出预留
自动压缩线 = 有效窗口 - 13000                          // ≈ 178k，约 89%
警告线 = 自动压缩线 - 20000
硬阻断 = context_window - 3000                        // 超过拒绝生成
```

8.3 压缩后的消息结构（"摘要 + 近期保留"如何拼装）

```
[boundaryMarker]   # 压缩边界标记（类型/压缩前 token 数）
[summary]          # 1 条摘要（伪装 user 消息，不污染 assistant 轮次）
[messagesToKeep]   # 近期原文尾部（tool_use/tool_result 已剥离）
[attachments]      # plan 等附件
```

近期保留段的选择规则（Rust 还原版实现）：

1. 按 **token 预算**（非条数）从尾部往前累加，预算用完即切；

2. **配对安全吸附**：切点若落在 `tool_result` 上则回退——宁多勿拆（API 要求 tool_result 紧跟 tool_use）；
3. 切点之前全部送**单次非 agentic API 调用**生成摘要（禁 thinking、输出上限 20k、极少工具），防止递归触发压缩。

8.4 压缩后什么存活（官方文档）

system prompt / output style 保留；**CLAUDE.md、auto memory、plan、skills 从磁盘重新注入**；git status 重读。设计哲学：**权威记忆落盘 + 每次重注入，上下文里的对话历史只是可丢弃缓存。**

8.5 若落地到本项目的演进路径

步骤	方案	成本	依赖
1	Microcompact 思路：超阈值先剥离旧轮次的工具结果 JSON	零 LLM 调用	无（消息结构已含 role 区分）
2	尾部 token 预算切分 + 配对吸附，头部单次 LLM 摘要替换	一次调用	步骤 1
3	压缩边界写入 SQLite `sessions` 表，刷新恢复时按边界拼装	无	步骤 2

当前不做：比赛规模下全量重放更稳，过早引入压缩是负资产。本章仅证明"想过、想清楚了"。

---

9. 演示与复现

7.1 启动（双服务）

```
# 终端 1：Python AI 接入层
pip install -r backend/requirements.txt
cp backend/.env.example backend/.env    # 填入 DEEPSEEK_API_KEY
python -m backend.app.rag.pipeline build
uvicorn backend.app.main:app --port 8000
```

```
# 终端 2：Next.js 前端
npm install
npm run dev
```

打开 http://localhost:3000。

7.2 演示脚本（对应视频分镜）

1. **RAG 问答**：问「淘宝 DSR 评分包含哪三项？」→ 展示 [1] 引用角标；
2. **Agent 模式**：问「查店铺今日数据并给建议」→ 展示步骤卡片 + 流式报告；
3. **图生文案**：上传商品图 → 识别卡片 → 标题自动填入；
4. **语音输入**：点 🎤 说话 → 文字上屏 → 发送；
5. **MCP**：终端运行 Inspector，调用 kb_search。

7.3 已知边界与后续方向

项	现状	方向
语料	40 篇方法论为主	扩充真实平台规则原文（数值型细节），提升 RAG 拉开差距的能力
data_query	样例数据	接入真实店铺数据源（ERP/API）
图表	无	data_query 结果 → ECharts 可视化
TTS	无	回答播报
安全	提示词级拒答	敏感词过滤、免责声明 UI
评测	通用问题为主	增加语料外私有数据的对比实验

附录：技术栈版本

层	技术	版本/说明
前端	Next.js / React / TypeScript	16.x / App Router / strict
样式	Tailwind CSS	v3

层	技术	版本/说明
AI SDK	Vercel AI SDK	v7（streamText / useChat）
后端	Python / FastAPI / uvicorn	3.12 / 0.115+
LLM	DeepSeek Chat	OpenAI 兼容接口
视觉	GLM-4V-Flash	智谱（可选）
嵌入	fastembed BGE-small-zh-v1.5	512 维，本地
向量库	LanceDB	嵌入式，零运维
分词	jieba	BM25 通道
MCP	mcp (FastMCP)	v1.x，stdio

文档版本 v1.1 · 更新于 2026-10-24 · 新增 §6 会话管理与引用溯源（P4）、§8 长会话与上下文压缩扩展设计